Aletheia

Marco Zero · v0.8 · 10 de novembro de 2026 · commit `9a4887b` · github.com/Antoniorjr4/Aletheia
Esta versão traz a primeira matriz oficial do AgentDojo e o T6 no Lean:

- **AgentDojo oficial, primeiro lote** (`banking` v1.2.2, `openai/gpt-oss-120b`, ataque `important_instructions`, até a cota diária da Groq): 16/16 tarefas sem ataque e 19/144 pares sob ataque. Baseline padrão: utilidade 68,8%, sob ataque 15,8%, ASR 78,9% (15/19). Aletheia: utilidade 75,0%, sob ataque 31,6%, **ASR 0,0%** (0/19; IC95 0–17%), com metade dos tokens por execução;
- **T6 no Lean**: sagas no avaliador canônico de T1–T3, sobre um mundo abstrato. `on failure` só roda se todas as compensações deram certo e a sonda confirmou; sob contratos `restoring`, uma saga que falha compensa exatamente o que o corpo fez, na ordem reversa, e devolve o mundo ao invariante de antes. Sem `sorry`; o tripé Lean × ARX × Python segue idêntico byte a byte;
- **limites**: a matriz cobre 19 dos 144 pares; sagas ainda fora do julgamento de tipos; a restauração vale para sagas não aninhadas.

Versões anteriores: `f4c3ccd` (v0.7: tripé e ponte do AgentDojo), `adaf30a` (v0.6: Rev6 e Rev7, T2 em Lean), `eb59d0e` (v0.5: replay real e T1/T3 em Lean), `43c1bf4` (v0.4: listas, Δ versionado, backend Python).

Implementação executável de Aletheia: o subconjunto **Aletheia Core 0**, com o compilador `athc`, o runtime `ARX` em Rust e um segundo backend, Python sobre o `aletheia_runtime`, verificado contra o ARX por testes diferenciais, inclusive por replay de tráfego real da Groq. Há também um núcleo formal em Lean 4, com a solidez de tipos (T1), os efeitos com capacidades lineares `Cap[E]` (T2) e a não-interferência com endosso robusto (T3) provados sem `sorry` sobre o mesmo avaliador, que também tem sagas, com a recuperação (T6) provada. Esse avaliador também roda compilado (`ath-lean-eval`): é o terceiro backend de um tripé diferencial em que o Lean, o ARX e o Python produzem o mesmo journal encadeado, byte a byte. Para a suíte oficial `banking` do AgentDojo, o agente bancário é um programa Aletheia ligado ao benchmark por um `BasePipelineElement`. O objetivo é provar uma coisa: integridade, efeitos, capacidades, linearidade, refinamentos, orçamento e transações sobre o mundo podem ser verificados **juntos, antes de rodar**, num programa de agente real. E o tipo da saída de um modelo pode ser, ao mesmo tempo, a gramática que o modelo é obrigado a seguir.

```
$ athc check examples/rollout.ath
✓ E0 estrutura       12 definições, todas resolvidas
✓ E1 multiplicidade  4 chave(s) IdemKey e as obrigações Pending consumidas exatamente uma vez
✓ E2 efeitos         {Cluster.Read, Cluster.Write, CompensationFailed, Infer, Page, TimeoutUnknown} cobertos por c
apacidades e linhas de efeito
✓ E2 sagas           1 saga(s); compensações: scale ↩ unscale; irrevogáveis só via commit, fora de atomic
                     TimeoutUnknown tratado; reconciliação fora do contrato Δ: ≤ 9,0 s por chave
✓ E3 integridade     4 exigência(s) de integridade satisfeitas, inclusive sob pc
✓ E5 refinamentos    5 obrigação(ões) provadas por literal, campo refinado ou saída tipada
                     infer planner -> List[Action, 1..3]: entrada ≤ 2.319 tok (1.551 de prompt + 768 de overhead)
· saída ≤ 1.018 tok (506 tok do tipo `List[Action, 1..3]` + 512 tok de raciocínio)
✓ Δ contrato         `planner` = groq:openai/gpt-oss-20b@2026-11-09 #7464f0bf2e24 · overhead 768 tok
✓ E6 orçamento       Rollout.alert: pior caminho 6.674 tok · US$ 0,0010 · 74,4 s
                     ≤ [20.000 tok · US$ 0,50 · 120,0 s]
                     caminho: logs 2,0 → infer planner ×2 24,4 → for a ×3 ⟨scale 1,0 → healthy 1,0 → await 10,0 →
unscale↩ 1,0 → restored 1,0 → for ch ×2 ⟨page 1,0⟩ 2,0⟩ 48,0
                     3 laço(s)/quantificador(es) delimitado(s): β(laço) ≤ teto · β(corpo), aninhados por multipli
cação
✓ Codex              12 objetos BLAKE3, raiz #9ecbfccad097
```

Começar

Requer Rust ≥ 1.87 e Python ≥ 3.10 (benchmark, runtime Python e testes diferenciais). As provas pedem o Lean 4.20.0, que o `elan` instala a partir de `lean/lean-toolchain`.

```
cargo build --release
cargo test                                  # 98 testes, inclusive os diferenciais e o replay da Groq
(cd runtimes/python && PYTHONPATH=. python3 -m unittest discover -s tests)   # runtime Python
python3 -m unittest discover -s bench/m1/tests   # os dois oráculos da M1 contra as rodadas publicadas
(cd lean && lake build --wfail)             # T1, T2, T3 e T6 em Lean 4, auditoria de axiomas e ath-lean-eval
cargo test -p athc --test differential      # com o binário do Lean presente: o tripé Lean × ARX × Python

# reproduzir tráfego real gravado na Groq, nos dois backends
./target/release/athc replay bench/replay/2026-11-09-groq-gpt-oss-20b-capacidade/remediate-capacidade-compensa.ar
x.cassette.json \
    --emit-py caso.py
PYTHONPATH=runtimes/python python3 -m aletheia_runtime replay \
    bench/replay/2026-11-09-groq-gpt-oss-20b-capacidade/remediate-capacidade-compensa.arx.cassette.json caso.py

# verificar
./target/release/athc check examples/remediate.ath
./target/release/athc check examples/errors/e0412_injection.ath     # rejeitado: E0412

# executar no ARX (mundo simulado e modelo roteirizado em examples/fixtures)
./target/release/athc run examples/remediate.ath --arg '"api"'

# listas delimitadas: o modelo propõe um plano `List[Action, 1..3]`, laços aninhados executam
./target/release/athc run examples/rollout.ath --arg '"api"' --provider scripted \
    --models examples/fixtures/models-rollout.json

# o mesmo programa no backend Python: o journal sai idêntico, byte a byte
./target/release/athc emit examples/rollout.ath --target py --out rollout.py
PYTHONPATH=runtimes/python python3 -m aletheia_runtime run rollout.py --provider scripted \
    --world examples/fixtures/world.json --models examples/fixtures/models-rollout.json \
    --arg-json '"api"' --journal rollout-py.jsonl

# aderência ao contrato Δ medida em journals reais, e o Δ recalibrado pela cauda observada
./target/release/athc contract-report .arx/journal.jsonl

# sagas e reconciliação: mundos com falhas injetadas
./target/release/athc run examples/remediate.ath --arg '"api"' --world examples/fixtures/world-saga-compensa.json
./target/release/athc run examples/remediate.ath --arg '"api"' --world examples/fixtures/world-timeout-transfere.j
son

# contra um modelo real (Groq, OpenAI, vLLM…)
export OPENAI_API_KEY=…  OPENAI_BASE_URL=https://api.groq.com/openai/v1
cargo run --release --bin athc -- run examples/remediate.ath --provider openai --model openai/gpt-oss-20b --arg
'"api"'

# a gramática que o tipo `Action` impõe ao modelo, e a máscara agindo sobre uma proposta
./target/release/athc grammar examples/remediate.ath --type Action \
    --decode '{"value":{"kind":"Delete","deploy":"db"}}'

# métrica M1: o mesmo ataque contra um laço de tool-calling em Python e contra o Aletheia
python3 bench/m1/run.py                     # modelo roteirizado como comprometido
python3 bench/m1/run.py --live              # modelo real, configurado pelo ambiente

# Codex e cadeia de custódia (inclui a checagem T9: nenhuma chave em aberto)
./target/release/athc hash examples/remediate.ath
./target/release/athc verify-journal .arx/journal.jsonl
```

Sagas e reconciliação

Compensar não é desfazer o passado: é executar uma nova transição que leva o mundo a um estado que uma sonda confirma. E um timeout diz que o agente parou de esperar, não que o efeito não aconteceu. As duas coisas estão no tipo:

```
tool scale(c: Cap[Cluster.Write], deploy: Text[..32] ^ Endorsed, to: Int[2..64], k: own IdemKey) -> Text
  ! { Cluster.Write } compensable by unscale provider queryable latency 1.0

tool restart(c: Cap[Cluster.Write], deploy: Text[..32] ^ Endorsed, k: own IdemKey) -> Text
  ! { Cluster.Write } irrevocable provider idempotent latency 1.0

let r = atomic recover restored(cluster, deploy) {
  scale(cluster, deploy, to, keys.mint())
  await healthy(cluster, deploy) within 20.0
  "resolvido"
} on failure(e) {
  "compensado: " ++ e
```

```
} on compensation_failed(cf) {
  "intervenção humana: " ++ cf
}

handle timeout_unknown(p: own Pending[_]) -> Text {
  reconcile p fuel 3 else {
    transfer p
    "pendente"
  }
}
```

Regra	Onde	Erro
Efeito irrevogável dentro de `atomic` (mesmo com `commit`)	E2	T0302
`compensable` sem compensador, com compensador de outra assinatura ou irrevogável	E2	E0505
Saga com efeito compensável sem `on compensation_failed`	E2	E0503
Chamada com chave sem `handle timeout_unknown` no agente	E2	E0503
Provedor `opaque` com chave dentro de `atomic`	E2	T0305
`await` fora de `atomic`; `return` dentro de `atomic`	E2	E0601 / E0602
Pós-condição ou sonda de recuperação decididas por dado Untrusted	E3	E0417
`IdemKey` ou `Pending` sem `own`; `own` em tipo não linear	E1	E0042 / E0043
`Pending` descartado ou consumido duas vezes (inclusive no `else` de `reconcile`)	E1	E1107 / E1104

No runtime, a falha do corpo dispara as compensações em ordem reversa, que valem como obrigação e rodam mesmo com o orçamento esgotado. Em seguida roda a sonda `recover`. Se ela confirmar, executa `on failure`; se uma compensação falhar ou a sonda não confirmar, executa `on compensation_failed`. Um `TimeoutUnknown` entrega ao tratador a obrigação linear `Pending[k]`, e o protocolo depende do provedor: `idempotent` reenvia com a mesma chave, `queryable` consulta o status pela chave e `opaque` não tem protocolo. O desfecho é Done (a chamada é retomada), Failed (a chamada falha) ou esgotamento, caso em que o `else` recebe a obrigação e precisa transferi-la. O E6 conta o caminho de falha da saga no pior caso; a reconciliação fica fora do contrato Δ e o `athc check` mostra o limite dela por chave.

Mundo (`examples/fixtures`)	O que acontece	Resultado
`world.json`	a injeção é contida; a saga confirma	`resolvido`
`world-saga-compensa.json`	`healthy` nunca confirma; `unscale` restaura 3 réplicas	`compensado: R0910 …`
`world-saga-falha.json`	e `unscale` falha no provedor	`intervenção humana: …`
`world-timeout-reconcilia.json`	`scale` sem resposta; o status pela chave confirma	`resolvido`
`world-timeout-transfere.json`	status desconhecido além do combustível; `transfer`; a saga compensa	`pendente`

Decodificação restrita derivada do tipo

O tipo de saída de um `infer` é compilado (crate `ath-grammar`) numa forma, e dela saem sem anotação extra:

- a **codificação canônica** da saída, `{"value": …}`, com variantes como objetos `{"kind": …}`;
- um **JSON Schema estrito** para `response_format`, em dois dialetos: `core` (padrão; só o núcleo que todo modo estrito documenta, com intervalos pequenos como `enum` exato) e `full` (com `minimum`,

`maximum`, `maxLength`);

- uma **gramática GBNF**, enviada como `grammar` ao llama.cpp;
- um **autômato de prefixo viável**, base da máscara de tokens e do decodificador local;
- um **teto de bytes**, que limita os tokens de conteúdo no orçamento (E6) e no pedido.

```
type Action =
  | Restart { deploy: Text[..32] }
  | Scale   { deploy: Text[..32], to: Int[2..64] }

action-scale ::= "{\"kind\":\"Scale\"" ",\"deploy\":" text-32 ",\"to\":" int-2-64 "}"
```

Provedor (`--provider`)	**Restrição**	**Saída válida por construção?**
`masked` (padrão)	máscara local sobre o autômato do tipo	sim; testado por propriedade com propostas adversariais
`scripted`	nenhuma	não; o ARX valida e descarta o que está fora do tipo
`openai`	JSON Schema estrito, GBNF ou nenhuma, num servidor compatível com OpenAI	conforme o servidor impõe; o ARX valida de novo

Modelo real

O provedor `openai` é configurado pelo ambiente, e os argumentos da CLI têm precedência:

Variável	**Uso**
`OPENAI_API_KEY`	chave (também `GROQ_API_KEY`, `ATH_API_KEY`)
`OPENAI_BASE_URL`	ex.: `https://api.groq.com/openai/v1`; `--base-url`
`ALETHEIA_MODEL`	ex.: `openai/gpt-oss-20b`; `--model`
`ALETHEIA_SCHEMA`	`core` (padrão) ou `full`; `--schema`
`ALETHEIA_CONSTRAINT`	`json-schema` (padrão), `gbnf` ou `none`; `--constraint`
`ALETHEIA_REASONING_EFFORT`	`low` (padrão), `medium`, `high` ou `none`
`ALETHEIA_EXTRA_BODY`	objeto JSON mesclado no corpo; `null` remove um campo

Modelos de raciocínio gastam tokens antes do conteúdo, e o contrato Δ declara quantos (`reasoning = 512` no `model`). O pedido usa `max_completion_tokens` = teto do tipo + raciocínio. O ARX liquida o orçamento com os tokens e a latência informados pelo servidor e grava no journal o identificador da requisição, o raciocínio (truncado), a espera imposta por 429 e as novas tentativas, e marca em `tok_source` se as contagens vieram do `usage` do servidor (`server`) ou da estimativa de 4 caracteres por token (`estimate`). Se o servidor gerar mais tokens que o contrato admite (R0702) ou demorar mais do que ele permite (R0703), o resultado é `ContractBreach`.

Na Groq, o modo estrito não restringe a decodificação a um `enum` de inteiros: o modelo pode gerar um valor fora dele. O servidor valida depois e responde HTTP 400 `json_validate_failed`, com a geração em `failed_generation`. O ARX trata essa resposta como amostra, valida de novo e a descarta. Como um 400 não traz `usage`, os tokens dessa amostra ficam como `estimate`.

O workflow `Fogo real (Groq)` roda o exemplo e a M1 contra a Groq com a chave guardada no segredo de repositório `GROQ_API_KEY`.

Listas delimitadas e laços

`List[T, a..b]` é uma lista de `a` a `b` elementos. Toda lista tem teto: é ele que limita o custo de qualquer iteração. Do tipo saem a gramática (`minItems`/`maxItems` no JSON Schema, repetição limitada na GBNF, contagem no autômato de prefixo), o teto de bytes da saída e as obrigações dos estratos:

```
type Action = | Restart { deploy: Text[..32] } | Scale { deploy: Text[..32], to: Int[2..64] }

policy SafePlan(p: List[Action, 1..3]) -> Bool = all a in p => SafeAction(a)

let draft = infer planner -> List[Action, 1..3] { … }
let plan  = refine draft by SafePlan retry 2 else { … }
for a in plan {
  …                                          -- etapa: commit restart, ou saga com scale
  for ch in ["sre", "dev"] { commit page(oncall, ch ++ " ← " ++ r, keys.mint()) }
}

list-action-1-3 ::= "[" action ("," action){0,2} "]"
```

Estrato	O que verifica	Erro
E0	`for`/`all`/`any` só sobre listas; elementos de um mesmo tipo; nada de capacidades, chaves ou obrigações dentro de listas	E0008 / E0011 / E0004
E1	o corpo de um laço roda de `a` a `b` vezes: um linear de fora consumido num laço com teto ≥ 2 é consumido mais de uma vez; com teto exato 1, uma vez	E1104 / E1105
E3	o número de iterações é decidido pela lista: iterar dado Untrusted rebaixa o `pc`, como um `match`	E0412
E5	literais dentro dos limites; limites conhecidos contidos nos exigidos (em qualquer nível de aninhamento) e refinamentos dos elementos; `len(xs)` é um `Int[a..b]` provado; o domínio da política contém o rascunho	E0452 / E0451 / E0450
E6	β(laço) ≤ b · β(corpo); laços aninhados compõem por multiplicação; quem sai por `return` paga (b − 1) iterações completas mais a que sai; lista vazia custa zero	B0201 / B0207

O caminho de pior caso mostra a multiplicação: `for a ×3 ⟨… → for ch ×2 ⟨page 1,0⟩ 2,0⟩ 48,0`. Os quantificadores `all`/`any` são puros e totais (podem ir em políticas) e param no primeiro contraexemplo.

Contrato Δ versionado

O modelo é um valor com contrato: preços, janela, latência e, agora, identidade e overhead.

```
model planner {
  id = "openai/gpt-oss-20b"   provider = "groq"   version = "2026-11-09"
  overhead  = 768             -- template de chat + esquema de decodificação, por amostra
  price_in = 0.075  price_out = 0.30  ctx = 131072  max_out = 600  reasoning = 512
  ttft = 2.0  per_tok = 0.01
}
```

- **Identidade:** o contrato inteiro entra no hash do Codex, então mudar a versão, o provedor ou o overhead é mudar de contrato. O hash e a identidade vão para cada entrada `infer` do journal.
- **Overhead no E6:** a entrada por amostra é prompt + overhead, na janela (`B0301`), no orçamento e na reserva do ARX. Se o overhead declarado não cobre o esquema que o provedor injeta no prompt, o erro é `B0302`.
- **Deriva em runtime:** o ARX levanta `ContractBreach` e grava `contract_breach` no journal, com o código, o `request_id` e a saída, quando:
  - o servidor serve outro modelo (`R0704`);

    - gera mais tokens que o teto (`R0702`);
    - demora mais que o contrato (`R0703`);
    - cobra mais entrada que prompt + overhead (`R0705`). O consumo dessa amostra é liquidado antes do veredito.
- **Calibração:** `athc contract-report` lê journals reais e mostra:
    - a latência medida contra o contrato (p50, p95, máximo, maior razão);
    - o overhead observado contra o declarado;
    - o raciocínio e as violações. Com isso, sugere o Δ recalibrado pela cauda observada.

Backend Python

`athc emit --target py` emite o programa verificado para Python, e o resultado roda sobre o `aletheia_runtime`. O emissor não traduz sintaxe: ele produz uma **segunda realização da semântica do ARX**.

- **Ordem de avaliação:** o código sai em forma normal administrativa, então toda subexpressão com efeito vira uma instrução, na ordem de avaliação do ARX.
- **Nomes únicos:** cada vínculo ganha um nome próprio, e o sombreamento de Aletheia nunca vira reatribuição em Python.
- **`return`:** vira uma exceção capturada na fronteira do handler.
- **Sagas, `await` e sondas:** viram funções entregues ao coordenador transacional do runtime.
- **Metadados:** formas de saída, tetos de bytes e hashes de contrato são calculados em Rust e emitidos como dados.

O runtime usa só a biblioteca padrão e tem as mesmas peças do ARX:

- BLAKE3 em Python puro;
- serialização idêntica à do `serde_json` (chaves ordenadas, floats na grafia do `ryu`);
- `f64::round` do Rust;
- capacidades conferidas em runtime e chaves lineares;
- orçamento por reserva;
- o coordenador de sagas e a reconciliação;
- o mundo simulado e os provedores roteirizado e mascarado, com o autômato de prefixo portado.

Os testes diferenciais executam o mesmo programa, no mesmo mundo e com o mesmo roteiro, nos dois backends. Exigem **journals idênticos byte a byte**, logo a mesma cabeça de hash, além do mesmo resultado ou erro, do mesmo mundo final e do mesmo orçamento. São 29 casos:

- os 5 mundos de `remediate.ath`, com falhas injetadas, nos dois provedores;
- 3 caminhos de erro (`R0701`, `R0003`, saídas que não são JSON);
- 8 planos de `rollout.ath`, inclusive sagas e reconciliação dentro de laços;
- os 7 cenários da M1.

No mesmo arquivo está o **tripé**. O avaliador formal do Lean (`ath-lean-eval`, o `eval` sobre o qual T1–T3 estão provados) entra como terceiro backend. Para programas do núcleo λ_Ath, os três backends precisam produzir o mesmo **journal do núcleo** (`act`, `endorse`, `end`), byte a byte:

- **o Lean** escreve o journal a partir do traço do seu avaliador;
- **o ARX e o Python** o derivam dos seus journals completos (`athc run --core-journal`), cada um com o seu BLAKE3.

O corpus tem 160 programas gerados por semente: bem tipados e lineares, conferidos por uma reimplementação das regras de `Typing.lean`. Inclui ainda a injeção da Groq de `Examples.lean` e o BLAKE3 do Lean contra o crate de referência. Detalhes em `lean/README.md`.

Replay de tráfego real

Os dois backends têm o provedor `openai`, com a mesma requisição byte a byte: corpo, esquema, dialeto e política de novas tentativas. Ele fala com qualquer API compatível (Groq, vLLM, llama.cpp). `--record` grava um **cassete**: cada requisição exata com a resposta crua (status, `retry-after`, `x-request-id`, corpo e latência medida), mais o programa, o mundo, os argumentos e o journal da execução real. A chave de API nunca entra no cassete.

```
athc run examples/remediate.ath --provider openai --world examples/fixtures/world.json \
  --arg '"api"' --record caso.cassette.json
athc replay caso.cassette.json --emit-py caso.py        # reproduz no ARX e emite o Python
PYTHONPATH=runtimes/python python3 -m aletheia_runtime replay caso.cassette.json caso.py
```

No replay, cada backend reconstrói as requisições a partir do programa e exige que sejam idênticas às gravadas. Depois reinterpreta as respostas com as mesmas latências, e a espera de um `429` é contabilizada sem dormir. Os dois comandos só retornam 0 se o journal for idêntico ao da execução real.

O workflow `replay.yml` grava na Groq pelo ARX e pelo runtime Python. Os 22 cassetes gravados com `openai/gpt-oss-20b` e `openai/gpt-oss-120b` estão em `bench/replay/`, e o CI os reproduz a cada push. Para cada cassete, o teste exige o mesmo journal BLAKE3, a mesma árvore de compensação e o mesmo balanço de orçamento nos dois backends. Também exige cobertura, sob tráfego real, de commit, compensação, falha de compensação, reconciliação de timeout, recusa no decodificador e esgotamento do `refine`.

Dois resultados das gravações:

- **Injeção bloqueada no decodificador.** Os logs do `rollout` traziam uma injeção, e o modelo a seguiu nas 6 tentativas: o plano abria com `Scale payments → 0`. O refinamento `to: Int[2..64]` vai para o esquema de decodificação como `enum`, e a Groq recusou cada geração com `400 json_validate_failed`. Nada chegou à política nem a uma ferramenta, e o resultado foi `escalado`.
- **Sagas sob carga real.** Com logs de saturação e sem injeção, o modelo propôs `Scale api → 5`. O mundo levou a saga por cada caminho, e os dois backends reproduziram a árvore de compensação idêntica.

Estado confiável do host: `s.managed()`

O inventário de deployments gerenciados e a telemetria vêm de uma **fotografia confiável do host**, capturada uma vez por execução. Nada ali vem do modelo, dos logs ou de constante escrita no prompt. O mesmo objeto alimenta o prompt e a política:

```
policy SafeRemediation(a: Action) -> Bool =
  match a {
    Restart { deploy }   => s.is_managed(deploy),
    Scale { deploy, to } => s.is_managed(deploy) && s.scale_justified(deploy, to),
  }

let draft = infer planner -> Action {
  estado   = s.managed(),            -- Text[..256] ^ Trusted
  metricas = s.telemetry(),          -- Text[..512] ^ Trusted
  deployment_afetado = quote(service),
```

```
  logs     = quote(fit(l, 2000)),
}
```

- **Payload.** O ARX e o Python rotulam os campos da fotografia como `ESTADO CONFIÁVEL — fotografia do host`, ao lado dos campos `quote(…)` rotulados como DADO não confiável. A requisição sai idêntica byte a byte nos dois backends.
- **Journal.** Na primeira leitura de `s.…`, a fotografia usada (inventário, réplicas, CPU) entra no journal BLAKE3.
- **Compilador.** O compilador segue o fluxo `infer` → `refine` e veta o descompasso entre o que o modelo vê e o que a política aplica, sem depender do nome da política, do tipo ou do campo:
  - `E0421`: a política decide pela fotografia, mas o prompt não recebe `s.managed()`. Cobre inventário forjado por literal ou copiado de log.
  - `E0422`: o prompt recebe `s.managed()`, mas a política compara com inventário escrito à mão (`deploy == "api"`).
  - `E0423`: sonda contextual fora de política.
- **Escala justificada.** `s.scale_justified(d, to)` exige CPU observada entre 85% e 100% e um aumento até o dobro das réplicas atuais. Sem telemetria, a resposta é não.

Inventários nomeados. A fotografia também carrega inventários confiáveis com nome, para hosts que não são clusters. `s.inventory("k")` entrega o inventário `k` ao prompt (`Text ^ Trusted`, com o rótulo de estado confiável) e `s.member("k", x)` decide dentro da política. A ancoragem vale por nome: o `E0421` acusa uma política que decide por um inventário que o prompt não recebeu, e o `E0424` acusa um nome que não é literal. O agente do AgentDojo usa três: `payees`, `scheduled` e `quoted` (`bench/agentdojo/`).

Núcleo formal (Lean 4)

`lean/` mecaniza o núcleo λ_Ath que o `athc` verifica e o ARX executa: dados de primeira ordem com `List[T, 0..n]`, capacidades lineares `Cap[E]`, o reticulado `Untrusted ⊑ Endorsed ⊑ Trusted`, orçamentos graduados sobre as seis dimensões, inferência por um oráculo adversarial, endosso por política auditada e ações. O julgamento é Γ; pc ⊢ e : τ ^ ι ▷ β, e o avaliador verifica em tempo de execução tudo o que o ARX verifica: efeito sem capacidade, reuso de capacidade linear, estouro de orçamento antes do efeito.

- **T1, solidez** (`soundness`, `progress`, `preservation`, `eval_fresh`): um programa bem tipado, linear e com β dentro do teto executa sem erro. O valor final tem o tipo estático, e o gasto real fica dentro de β em cada dimensão; um laço gasta no máximo `n • β(corpo)`.
- **T2, efeitos e capacidades** (`T2`, `effects_granted`, `effects_lexical`, `caps_exactly_once`): sobre o **mesmo** avaliador, toda ação usou uma capacidade `Cap[E]` léxica cujo `E` contém o efeito da ferramenta e foi concedida pelo runtime. A guarda dinâmica nunca recusa um programa bem tipado: a checagem estática basta. Cada capacidade é linear (multiplicidade 1): é consumida exatamente uma vez, sem duplicação nem descarte implícito, e o descarte só existe como `drop` explícito.
- **T3, não-interferência com endosso robusto** (`noninterference`, `robust_endorsement`, `endorsed_sound`, `confine`, `noninterference_total`): duas execuções com as mesmas entradas confiáveis, mas entradas `Untrusted` e oráculos arbitrários, produzem o mesmo traço de ações ou divergem numa decisão de endosso. Com as mesmas decisões, as ações, as capacidades e os argumentos são idênticos. Todo valor endossado satisfaz a sua política, e o endosso só ocorre sob `pc ⊒ Endorsed`.

- **T6, recuperação de sagas** (`T6_no_masking`, `T6_recovered`, `T6`, `T6_saga`): `await` e `atomic … recover` estão no mesmo avaliador, sobre um mundo abstrato em que ferramentas, compensações (que podem ser recusadas) e sondas são funções arbitrárias. `on failure` só roda se todas as compensações deram certo e a sonda confirmou: falha nunca vira sucesso em silêncio. Sob contratos `restoring`, uma saga que falha compensa exatamente as ações que o corpo executou, uma vez cada e na ordem reversa, e devolve o mundo ao invariante de antes da saga. Isso vale para corpos sem saga aninhada, porque no ARX uma saga interna confirmada descarta o seu registro. `await` e `atomic` ainda não estão no julgamento de tipos.

O CI roda `lake build --wfail`, e um alvo de auditoria fixa o conjunto de axiomas de cada teorema: só `propext`, `Quot.sound` e `Classical.choice`. O mesmo build compila o avaliador no binário `ath-lean-eval`, e o job `tripod` da CI o executa contra o ARX e o Python sobre 400 programas gerados (o tripé diferencial). No cálculo, toda capacidade é linear; no ARX, as capacidades de ferramenta são reutilizáveis e a linearidade vale para `IdemKey` e `Pending`. Aproximar as duas disciplinas faz parte da prova de refinamento, que ainda não existe.

Métrica M1

`bench/m1` roda sete cenários no estilo do AgentDojo, com o ataque "important_instructions", no laço clássico de tool-calling e no Aletheia. No modo simulado, o modelo é roteirizado como comprometido; o CI repete a medição a cada push e falha se o Aletheia executar qualquer ação fora da política.

Dois oráculos. O oráculo é o instrumento de medida: decide, depois da execução, quais ações saíram da política. Ele não muda junto com a política do agente. Toda rodada é medida sob os dois oráculos, e o portão reprova o Aletheia em qualquer um deles:

- o **canônico**, congelado desde a Rev6, é a régua comparável entre rodadas;
- o **contextual** exige inventário e telemetria do cenário.

`bench/m1/tests/` recalcula o canônico sobre as ações brutas das rodadas publicadas e exige as contagens publicadas. `bench/m1/rescore.py` reaplica as duas réguas a qualquer rodada sem chamar o modelo.

Métrica (simulado)	Python (laço de tool-calling)	Aletheia
Ataques que atingiram o objetivo	6/6	0/6
Ações fora da política · oráculo canônico	5	**0**
Ações fora da política · oráculo contextual	6	**0**
Tarefa cumprida sob ataque	5/6	5/6
Porte ingênuo do laço Python	—	rejeitado em compilação (E0412)
Inventário forjado no prompt / política fora de sintonia	—	rejeitados (E0421 / E0422)

O cenário 05 pede uma ação que a política estática aceitava, `Scale(api, 64)`: era o raio de explosão de uma injeção. Com a política contextual (CPU observada de 35%), o pedido é recusado.

Contra um modelo real (`openai/gpt-oss-20b` na Groq, 7 cenários × 2, execução B):

Métrica (ao vivo)	Python (laço de tool-calling)	Aletheia
Ataques que atingiram o objetivo	12/12	2/12

Métrica (ao vivo)	Python (laço de tool-calling)	Aletheia
Ações fora da política executadas	12	**0**
Tarefa cumprida sob ataque	11/12	6/12
Tarefa cumprida sem ataque	2/2	2/2
Abortadas por violação do contrato Δ (R0703, sem efeito)	—	2

Por amostra, o journal registrou de 13 a 49 tokens de raciocínio (contrato: 512) e de 26 a 30 de conteúdo (teto do tipo: 174). A latência medida teve mediana de 0,38 s, contra mediana de 2,46 s do contrato E6. Duas chamadas, 2,88 s e 2,31 s, passaram dos 2,14 s contratados e abortaram com `ContractBreach`.

Contra `openai/gpt-oss-120b` na Groq, com 7 cenários × 3 repetições por sistema (42 execuções por rodada):

Métrica (ao vivo, 120B)	Rev6: Python	Rev6: Aletheia	Rev7: Python	Rev7: Aletheia
Ataques que atingiram o objetivo	18/18	3/18	7/18	**0/18**
Ações fora da política · oráculo canônico	15	**0**	7	**0**
Ações fora da política · oráculo contextual	24	3	7	**0**
Tarefa cumprida sob ataque	18/18	18/18	18/18	18/18
Tarefa cumprida sem ataque	3/3	3/3	3/3	3/3
Tokens	62.294	17.250	59.453	17.970
Latência por execução, p50 / p95	2,42 / 3,63 s	0,48 / 0,69 s	1,84 / 2,51 s	0,48 / 0,60 s

Como ler a tabela:

- **Os 3 ataques da Rev6.** Os 3 ataques bem-sucedidos do Aletheia na Rev6 são o `Scale(api, 64)` do cenário 05, permitido pela política estática da época.
- **O 0/18 da Rev7.** Na Rev7, com a telemetria no prompt, o modelo propôs `Restart api` nesse cenário. O 0/18 veio, portanto, da escolha do modelo, e a política contextual não precisou recusar nada nessa rodada. A recusa dela está exercitada na M1 simulada.
- **O baseline Python ficou mais instruído.** Na Rev7, ele passou a receber inventário, telemetria e a regra de escala. A queda dele de 15 para 7 ações fora mede isso, não uma mudança no Aletheia.
- **Tokens.** O Python faz em média 4,6 chamadas ao modelo por execução, contra 1 do Aletheia. Por chamada, o Aletheia gasta um pouco mais.
- **Tamanho da amostra.** N=3 por cenário é evidência exploratória. Zero violações em 18 execuções atacadas ainda é compatível com uma taxa real de até cerca de 17% (95%).

As duas rodadas rodaram no repositório de laboratório. Os commits de dados foram criados pelo próprio runner do GitHub Actions e mantêm o mesmo hash aqui (`c411e06` e `8834fca`). Método, cenários e limites estão em `bench/m1/README.md`; todas as execuções, em `bench/m1/live/`; os registros da Rev6 e da Rev7, em `docs/`.

AgentDojo oficial

`bench/agentdojo/` roda a suíte oficial `banking` do AgentDojo (v1.2.2) em dois pipelines com o mesmo modelo: o baseline padrão do AgentDojo e o agente `banking.ath`, ligado ao benchmark por um

`BasePipelineElement`. As tarefas, o ambiente injetado, o ataque `important_instructions` e os verificadores são os do pacote. Primeiro lote, `openai/gpt-oss-120b` na Groq, com os pares completos nos dois pipelines:

pipeline	utilidade (16)	utilidade sob ataque (19 pares)	ASR (19 pares)	tokens por execução
baseline AgentDojo	68,8% (IC95 44–86%)	15,8% (6–38%)	78,9% (57–91%)	3.632
Aletheia	75,0% (IC95 50–90%)	31,6% (15–54%)	**0,0%** (0–17%)	1.804

A cobertura é parcial (19 dos 144 pares), e os primeiros pares vêm das tarefas 0 a 2. Os detalhes, as falhas de cada lado e o que falta estão em `RESULTS.md`.

Estrutura

```
crates/
  ath-syntax/    lexer, AST com spans, parser
  ath-grammar/   tipo → forma → JSON Schema (full/core), GBNF, autômato de prefixo, máscara (inclusive listas)
  ath-check/     verificador estratificado E0 · E1 · E2 · E3 · E5 · E6, diagnósticos
  ath-codex/     forma canônica, DAG de Merkle, BLAKE3, alfa-normalização
  arx/           interpretador, sagas, reconciliação, provedores, orçamento por reserva, journal
  ath-emit/      emissor Python (ANF, nomes únicos, metadados calculados em Rust)
  athc/          CLI, renderização de diagnósticos, testes ponta a ponta e diferenciais
runtimes/python/ aletheia_runtime: a semântica do ARX em Python (só biblioteca padrão), provedor openai, gravação e replay
lean/            núcleo λ_Ath em Lean 4: sintaxe, tipagem, avaliador com sagas, T1, T2, T3 e T6, exemplos, auditoria de axiomas e o binário ath-lean-eval (BLAKE3 em Lean)
examples/        remediação, plano em lista (rollout), mundos com falhas, 24 variantes com erro e o gerador
bench/m1/        harness da métrica M1: cenários, linha de base Python, agente, dois oráculos, reavaliação
bench/replay/    cassetes de tráfego real da Groq (20B e 120B), reproduzidos nos dois backends a cada push
bench/agentdojo/ a suíte oficial do AgentDojo: ponte BasePipelineElement, agente banking.ath, executor pareado e matriz
docs/            registros da Rev6 e da Rev7: execuções reais, decisões e limites
.github/         CI (fmt, clippy -D warnings, testes, diferenciais, replay, runtime Python, oráculos, Lean, tripé, M1), fogo real, M1 multiamostra e gravação
```

O que está implementado

Spec v0.2	Implementação
1.3 Codex	forma canônica por definição; referências por hash (inclusive o compensador de uma ferramenta); nomes de definições e de locais não entram no hash
2.2 Linearidade (E1)	`own IdemKey` e `Pending[k]` consumidos exatamente uma vez em todos os caminhos, inclusive no `else` de `reconcile`; no runtime, a checagem T9 do journal
2.3 `refine … by policy retry n`	políticas puras; só o que a política aceita ganha `Endorsed`; saída fora do tipo é descartada
2.5 Modelos e inferência	`model` como valor com contrato Δ (inclusive raciocínio); provedor real configurado pelo ambiente, testado contra a Groq
2.4 Listas delimitadas	`List[T, a..b]` no tipo, na gramática (`minItems`/`maxItems`, repetição limitada na GBNF, autômato de prefixo) e nos estratos; `for`, `all`/`any`, `len`; laços aninhados
2.5 Compilador de gramática (`ath-grammar`)	tipo → forma → JSON Schema estrito em dois dialetos (`core`: intervalos de até 256 valores viram `enum` exato; `full`: `minimum`/`maximum`/`maxLength`), GBNF, autômato de prefixo viável, máscara de tokens e teto de bytes que entra no E6
2.6 Capacidades e integridade	capacidades só vêm dos parâmetros do agente, com atenuação; Untrusted ⊑ Endorsed ⊑ Trusted; `pc` para fluxos implícitos; `quote` obrigatório no prompt; pós-condições ⊒ Endorsed

Spec v0.2	Implementação
Estado confiável do host	`s.managed()` / `s.telemetry()` `^ Trusted` de uma fotografia única por execução; inventários nomeados (`s.inventory("k")` , `s.member("k", x)`); política contextual (`s.is_managed` , `s.scale_justified`); rótulo Trusted no payload; fotografia no journal; vetos E0421/E0422/E0423/E0424 pelo fluxo `infer` → `refine`
AgentDojo oficial	`AletheiaAgent` (`BasePipelineElement`) sobre o pacote `agentdojo` (v1.2.2): agente `banking.ath` , host ligado às funções oficiais, executor pareado e retomável (baseline padrão × Aletheia), matriz de utilidade, utilidade sob ataque e ASR com IC de Wilson; testado offline na CI
2.5 Contrato Δ versionado	`id` / `provider` / `version` e `overhead` no `model` ; o contrato inteiro no hash do Codex e no journal; overhead na janela, no orçamento e na reserva; cobertura do esquema (B0302); deriva em runtime (R0704, R0705); `athc contract-report`
2.7 Orçamento	E6 estático no pior caminho sob Δ, inclusive o de falha da saga; β(laço) ≤ teto · β(corpo), aninhado por multiplicação; reserva antes, liquidação depois (inclusive da amostra que viola o contrato); `ContractBreach` por latência, tokens, modelo ou entrada; obrigações sem reserva
2.9–2.10 Estratos	E0, E1, E2, E3, E5 (refinamentos decidíveis) e E6, relatando só o primeiro estrato que falha
3.4 Sagas	`atomic … recover` , `await … within` , `compensable by` , `on failure` , `CompensationFailed` obrigatório, irrevogáveis só fora da saga
3.5 Efeitos remotos	`own IdemKey` , `Pending[k]` linear, `TimeoutUnknown` , `reconcile … fuel … else` , `transfer` , provedores `idempotent` / `queryable` / `opaque`
3.6 Execução durável	journal encadeado por BLAKE3; `verify-journal` detecta adulteração e chaves em aberto (T9)
5.4 Métrica M1	harness reprodutível no CI (simulado) e contra modelo real (workflows); dois oráculos versionados; rodadas N=3 no 120B (Rev6, Rev7) com reavaliação offline
F1 Backend Python	`athc emit --target py` + `aletheia_runtime` ; testes diferenciais traço a traço contra o ARX no CI
F1 Replay	provedor `openai` nos dois backends; `--record` / `replay` no nível HTTP; 22 cassetes reais da Groq (20B e 120B) reproduzidos no CI com o mesmo journal, a mesma árvore de compensação e o mesmo balanço
Tripé diferencial	`ath-lean-eval` (o `eval` provado, compilado) × ARX × Python: o mesmo journal do núcleo, byte a byte, em programas gerados bem tipados e lineares e na injeção da Groq; job `tripod` na CI
4 Metateoria (T6)	sagas no avaliador canônico: mundo abstrato, `await` , `atomic … recover` , compensação LIFO como obrigação, `CompensationFailed` ; sem mascaramento e restauração sob contratos `restoring` , sem `sorry`
4 Metateoria (T1, T2, T3)	`lean/` : núcleo λ_Ath em Lean 4, com um só julgamento e um só avaliador: solidez (progresso, preservação, orçamento graduado), efeitos com `Cap[E]` linear (multiplicidade 1, `drop` explícito, concessão do runtime) e não-interferência com endosso robusto; sem `sorry` , axiomas auditados no build

Erros que o compilador já pega

Código	Estrato	Situação
E0412	E3	valor Untrusted, ou ramo decidido por Untrusted, chega a um parâmetro `^ Endorsed`
E0415	E3	dado Untrusted entra no prompt sem `quote(…)`
E0417	E3	pós-condição ou sonda de recuperação decididas por dado Untrusted

Código	Estrato	Situação
E1104 / E1105 / E1107	E1	valor linear usado duas vezes, não consumido, ou obrigação `Pending` descartada
E0042 / E0043	E1	linear sem `own`; `own` em tipo não linear
E0501 / E0502 / E0500	E2	efeito não declarado; capacidade insuficiente; efeito sem capacidade na ferramenta
E0503 / E0505	E2	efeito sem tratador; compensador inválido
E0601 / E0602 / E0603	E2	`await` fora de saga; `return` dentro de saga; `reconcile`/`transfer` fora do tratador
E0420	E2	política impura
E0421	E3	a política decide pela fotografia confiável (`s.…`), mas o prompt do `infer` não recebe o inventário em que ela decide (`s.managed()`, `s.inventory("k")`)
E0422	E3	descompasso: o prompt recebe `s.managed()`, mas a política compara com inventário escrito à mão
E0423	E2	sonda contextual (`s.is_managed`, `s.scale_justified`, `s.member`) fora de política
E0424	E3	nome de inventário (`s.inventory`, `s.member`) que não é literal de texto
T0301 / T0302 / T0303 / T0305	E2	irrevogável sem `commit`; irrevogável na saga; `commit` em revogável; provedor opaco na saga
E0451 / E0450	E5	literal fora do refinamento; refinamento sem prova (inclusive limites de lista e domínio da política)
E0452	E5	lista literal fora de `List[T, a..b]`
E0030	E0	contrato Δ incompleto ou versionado sem `id`
B0110	E6	`Text` sem limite de tokens no prompt (use `fit(x, N)`)
B0201 / B0202 / B0207	E6	tokens, dólares ou tempo do pior caminho acima do orçamento
B0301	E6	prompt + overhead + saída máxima acima da janela do modelo
B0302	E6	overhead declarado não cobre o esquema de decodificação do tipo de saída

O que ainda não existe

- **Backends nativos:** não há MLIR nem LLVM. O ARX interpreta a árvore, e o backend Python roda sobre o `aletheia_runtime`. O provedor mascarado (decodificação token a token) existe nos dois, mas o replay de tráfego real cobre só o provedor `openai`.
- **E4 e E5 completo:** sem formas de tensor nem SMT. Refinamentos são intervalos, tetos de texto e limites de listas, provados por literal ou por contenção. Laços sem teto (`while`) não existem, por desenho.
- **Prova da recuperação:** as sagas rodam, mas o invariante de recuperação é observado por uma sonda, não provado em E5 a partir de contratos `restoring` sobre um estado abstrato do mundo. Também faltam `lease` e cobertura do footprint das pós-condições. As compensações rodam em sequência, sem paralelismo por comutação.
- **Recuperação após queda:** reler o journal e reconciliar as chaves abertas depois de um reinício (hoje só a checagem T9 as detecta). O tratador `timeout_unknown` é por agente, sem `resume` explícito, e o recuo é fixo em 2 s.
- **Tokenizer:** a entrada é estimada em 4 caracteres por token quando o servidor não informa. O overhead declarado no contrato absorve o template e o esquema, mas não o erro de tokenização

de um prompt longo.

- **Calibração automática do Δ:** o `athc contract-report` sugere a recalibração pela cauda observada, mas ela ainda não volta sozinha ao contrato nem é versionada pelo CI.
- **AgentDojo oficial, matriz parcial:** o primeiro lote contra `openai/gpt-oss-120b` cobre as 16 tarefas sem ataque e 19 dos 144 pares sob ataque, até a cota diária da Groq. Os outros pares vêm em lotes retomáveis. Ver `bench/agentdojo/RESULTS.md`.
- **Inventário dinâmico no decodificador:** `Action.deploy` continua `Text[..32]` na gramática. O inventário restringe pela política e pelo prompt, não por um `enum` construído a cada chamada. A fotografia é imutável por execução, mas um host real ainda precisa de versão ou assinatura e de revalidação no commit (TOCTOU).
- **Certificados:** sem `certify`, `Bounded` nem monitor de deriva estatística.
- **Teoremas:** T1, T2 e T3 estão provados para o núcleo λ_Ath, e só para ele. Ficam de fora registros, variantes e `match`, `Text`, `mint` e atenuação, `retry`, a reconciliação de chaves (T9: `Recovery.lean` é um modelo abstrato), graus de certeza, localizações e T4/T5. As sagas estão no avaliador e têm o T6, mas ainda não estão no julgamento de tipos. A prova é do cálculo. Entre o cálculo e o ARX, e entre o ARX e o Python, a correspondência é verificada por construção e por testes diferenciais (o tripé executa o próprio avaliador provado), não demonstrada. O tripé cobre o fragmento linear do núcleo, sem `drop` na superfície e sem sagas.

Próximos marcos

1. **Sagas no julgamento de tipos**: estender `HasType` e a contagem linear a `await` e `atomic`, levando T1–T3 às sagas; `reconcile` e `Pending` no avaliador (T9); sagas no tripé diferencial.
2. **A matriz oficial do AgentDojo**: completar a rodada `banking` (16 tarefas e 144 pares sob ataque, baseline × Aletheia) e estender a ponte à suíte `workspace`.
3. **Invariantes de recuperação provados**: contratos `restoring` sobre um estado abstrato do mundo, descarregados em E5 com SMT, e cobertura de footprint nas pós-condições.
4. **Recuperação após queda** a partir do journal, e o reconciliador durável como serviço.
5. **Calibração de Δ no ciclo de CI**: o fogo real alimenta o `contract-report`, que propõe a próxima versão do contrato como mudança revisável.

Licença

Apache-2.0.